

CAVES: CONSTRAINED, ACCEPTANCE-AWARE SPECULATIVE DECODING WITH CASCADED VERIFICATION AND ALPHA-RATE CONTROL

Anonymous authors

Paper under double-blind review

ABSTRACT

Speculative decoding accelerates autoregressive generation but its speedups collapse when the acceptance rate α falls under high temperature or top-p settings, repetition or ban penalties, long or branchy drafts, or drift between draft and target models. Operators increasingly request α service-level agreements (SLAs) to stabilize tail latency, yet existing methods lack a calibrated acceptability predictor, a principled way to prune low-yield branches without changing the target model’s distribution, and a closed-loop controller that adapts draft shape under bandwidth and KV-cache constraints. We propose CAVES, a distribution-preserving framework that maximizes tokens per target-model call through three additions: (1) acceptance- and reward-aware drafting using a tiny acceptance estimator with temperature-isotonic calibration and split-conformal lower bounds, plus an optional tiny reward scorer for prioritization; (2) cascaded block verification that executes a cheap partial forward of the target model with candidate-only logits and an early-exit margin head to prune branches before full verification with KV reuse; and (3) a dual controller that enforces $\alpha \geq \alpha^*$ per request by adapting depth, width, draft temperature, and prune thresholds with uncertainty-aware safety. We detail an evaluation plan across calibration, compute savings, and SLA stability, and provide a prototype. An initial run aborted due to a deterministic cuBLAS configuration, so we report no numerical results and document remediation. CAVES complements prior speculative methods while preserving exactness and targeting predictable throughput on bandwidth-bound hardware ?????.

1 INTRODUCTION

Speculative decoding accelerates autoregressive generation by having a smaller draft model propose multiple tokens while a larger target model verifies them without changing the final distribution ?. Despite its promise, practical speedups are fragile: throughput collapses when the acceptance rate α drops, which happens under higher temperature or top-p, repetition and ban penalties, long or branchy drafts, or when the draft model diverges from the target. As deployments broaden and hardware increasingly becomes bandwidth-bound, operators seek α SLAs per request to stabilize tail latency across heterogeneous decoding configurations.

1.1 MOTIVATION AND GAPS

Three gaps impede SLA-oriented speculative decoding. First, stacks lack a cheap and calibrated acceptability predictor. Overconfident predictions directly cause SLA violations by expanding drafts when acceptance will be low. Second, there is no mechanism to prune low-yield branches early without altering the target model’s distribution; any pruning must preserve exactness. Third, serving systems lack a closed-loop controller that adapts draft depth, width, and draft temperature under bandwidth and KV-cache constraints to maximize useful work per target call while respecting an α constraint. Addressing these gaps spans calibrated prediction and uncertainty, dynamic computation, and constrained control ?????. Methods that alter logits at decode time can improve rewards but change the distribution, which is unacceptable when exactness is required ?.

1.2 PROPOSED APPROACH

We introduce CAVES: Constrained, Acceptance-aware, Verifier-Efficient Speculation. CAVES augments speculative decoding with three orthogonal components while preserving the target model’s distribution: (i) acceptance- and reward-aware drafting using a tiny acceptance estimator a_ϕ that predicts acceptance or the draft–target log-probability gap, calibrated via temperature scaling and isotonic regression with split-conformal one-sided lower bounds a_ϕ^{LB} , plus an optional tiny reward scorer r_ψ for prioritization; (ii) cascaded block verification with KV reuse that computes candidate-only logits and employs an early-exit margin head to prune before full verification while preserving exactness; and (iii) an alpha-rate constrained controller that adapts depth, width, draft temperature, and prune thresholds to maximize tokens per target call subject to $\alpha \geq \alpha^*$, with uncertainty-aware safety.

1.3 WHY THIS IS TIMELY

Serving is often bandwidth-bound; telemetry logs acceptance traces that enable supervised acceptability prediction and online calibration; micro-models under 10M parameters fit alongside the draft model with prefix and KV caching; candidate-only logits via selected output columns are practical; and demand for α SLAs and predictable tail latency is rising. CAVES aligns with these trends while enforcing exactness.

1.4 CONTRIBUTIONS

- **Acceptance-aware drafting:** Conservative, calibrated acceptability bounds and uncertainty gating increase expected accepted-prefix length without altering the target distribution.
- **Cascaded verification:** Candidate-only logits and a learned early-exit margin head prune node–token pairs safely while reusing KVs across stages.
- **Alpha-SLA control:** A decode-time controller adapts draft shape to maintain α while maximizing tokens per target call.
- **Theory and prototype:** A throughput model and an SLA guarantee under calibrated bounds and stable dual updates, plus an open prototype integrating training, calibration, kernels, and control.

We evaluate along three axes: (i) calibration and drafting gains, (ii) cascaded verification savings, and (iii) alpha-SLA stability. Comparisons include standard speculative decoding and speculative sampling ?. We discuss limitations of decode-time alignment for exactness ? and adhere to careful efficiency reporting ?. An initial execution aborted due to a deterministic cuBLAS configuration; we document remediation and withhold numerical claims. Beyond this paper, CAVES speaks to broader scaling and serving requirements in large language models ???.

2 RELATED WORK

2.1 SPECULATIVE DECODING AND SAMPLING

The seminal speculative decoding work demonstrates distribution-preserving acceleration by drafting with a smaller model and verifying with the target, showing $2\text{--}3\times$ speedups when acceptance is high ?. Speculative sampling modifies rejection sampling to better match hardware numerics while maintaining exactness ?. These techniques address the base algorithm but do not include calibrated acceptability prediction, uncertainty-aware pruning that preserves exactness, or a closed-loop α controller. CAVES augments these stacks by adding acceptance-aware drafting, conservative pruning, and rate-constrained control while keeping final sampling unchanged.

2.2 MASSIVELY PARALLEL VARIANTS

SpecExec builds large draft trees validated in single passes to exploit consumer GPUs and offloaded servers ?. It emphasizes throughput of tree construction and validation but neither predicts acceptability nor enforces α SLAs. CAVES is complementary: a_ϕ^{LB} and the cascaded verifier prune earlier,

and the controller stabilizes α under heterogeneous decode configurations. Integration preserves exactness by leaving verification unchanged.

2.3 DECODE-TIME ALIGNMENT

ARGS steers generation at decode time by adjusting logits using a reward model, removing the need for RLHF but changing the distribution ?. Because our focus is to preserve the target model’s distribution, ARGS is not an applicable baseline for exactness. In CAVES, the reward scorer r_ψ influences only drafting priority; acceptance and final sampling remain governed by the target model.

2.4 DYNAMIC COMPUTATION AND EARLY EXITS

Conditional computation and depth-adaptive Transformers adjust computation to input complexity ????. CAVES applies analogous ideas to verification: a partial forward with candidate-only logits and a margin head prunes work before full verification. Unlike dynamic networks that alter the main path, our pruning only limits which node–token pairs are fully verified, leaving the sampling distribution unchanged.

2.5 RATE-CONSTRAINED CONTROL AND SELECTION

Our controller maximizes useful work under an α constraint, connecting to implicit rate-constrained optimization where thresholds depend on parameters ? and to subset selection for maximizing expected order statistics under uncertainty ?. CAVES operationalizes these ideas with calibrated one-sided bounds and an uncertainty-aware node budget.

2.6 CALIBRATION AND TRUSTWORTHINESS

We adopt temperature scaling and isotonic regression with split-conformal prediction for one-sided coverage, consistent with trustworthy uncertainty estimation principles ?. We also refer to related viewpoints on speculative computation and decoding as dynamic programming for additional conceptual grounding ?. Finally, we follow guidance on reporting efficiency to avoid misinterpretation ?.

3 BACKGROUND

3.1 PROBLEM SETTING AND NOTATION

Let M_p denote the target language model and M_q a lighter draft model. At each decoding step, M_q proposes a draft tree of tokens, and M_p then verifies a path. A proposed token is accepted if M_p would have sampled the same token under the identical decode configuration; otherwise, the algorithm falls back to M_p ’s sampled token and proceeds. The acceptance rate α is the fraction of proposed draft tokens accepted by M_p at a step. Throughput depends on the expected accepted-prefix length of the drafted path per M_p call. In serving, α varies with temperature, top-p, penalties, and context; maintaining α above a target α^* stabilizes tail latency across heterogeneous workloads.

3.2 ACCEPTANCE PREDICTION AND CALIBRATION

We introduce a tiny acceptance estimator a_ϕ that, given the current prefix, features from M_q , and a candidate token, predicts either the acceptance probability or the log-probability gap $\Delta = \log p_{M_p}(x) - \log p_{M_q}(x)$. We calibrate a_ϕ via temperature scaling and isotonic regression and form one-sided lower bounds a_ϕ^{LB} using split-conformal quantiles. To prevent optimistic bias, we use asymmetric loss and Mondrian grouping over decode-config bins for coverage guarantees. These steps align with trustworthy calibration practices ?.

3.3 CASCADED VERIFICATION

We partition M_p into Stage A (layers $0 \dots L'$) and Stage B ($L' + 1 \dots L$). Stage A produces partial representations and KVs, computes candidate-only logits for the union of proposed tokens across

frontier nodes by selecting the corresponding output-projection columns, and evaluates a learned early-exit margin head. Combining margin thresholds with a_ϕ^{LB} prunes low-yield node-token pairs before full verification. Exactness is preserved because final acceptance remains identical to the baseline speculative procedure ?.

3.4 ALPHA-RATE CONTROL

We cast decoding as maximizing tokens per M_p call subject to $\alpha \geq \alpha^*$. A dual variable λ_A is updated from observed α ; actuators include tree depth γ , branching factor B , M_q temperature, and prune thresholds. Uncertainty from conformal intervals gates draft expansion. This connects to implicit rate-constrained optimization ? and to selection problems that maximize expected order statistics under uncertainty ?.

3.5 ASSUMPTIONS AND GUARANTEES

We assume availability of speculative logs with accept/reject labels and per-candidate features; micro-model budgets under 10M parameters for a_ϕ and r_ψ ; and kernel support for candidate-only projections and packed prefix attention. Exactness follows because Stage B performs standard speculative verification with M_p ; pruning only limits which candidates are verified. If a_ϕ^{LB} achieves one-sided coverage at least $1 - \delta$ and λ_A updates respect a stability bound, the steady-state probability of α violations is at most δ , with uncertainty gating further reducing transients, consistent with conservative uncertainty principles ?.

4 METHOD

Overview. CAVES augments any speculative decoding stack with three orthogonal components that preserve M_p 's distribution: acceptance- and reward-aware drafting, cascaded block verification with KV reuse, and an alpha-rate constrained controller. Final acceptance and sampling remain the standard M_p -based procedure ??.

4.1 ACCEPTANCE- AND REWARD-AWARE DRAFTING

Acceptance estimator a_ϕ . Inputs per candidate include M_q 's last-token hidden state, the candidate's embedding, q-logprob, top-2 margin, entropy, repetition and ban indicators, a decode-config embedding, a short α -history, and distance-to-eos. A two-head MLP under 5M parameters jointly predicts acceptance (via a logistic head) and Δ (via a regression head). Training uses class-weighted binary cross-entropy for acceptance and Smooth L1 for Δ . Post-hoc calibration applies temperature scaling and isotonic regression; split-conformal quantiles produce a one-sided lower bound a_ϕ^{LB} with target coverage. To discourage optimism, we employ asymmetric loss and optionally a ratio-matching auxiliary term when predicting Δ . Training randomizes M_q temperatures and top-p and includes domain embeddings to mitigate covariate shift.

Tiny reward scorer r_ψ . A lightweight per-token scorer under 10M parameters, distilled from preference data or teacher-forcing proxies ?, ranks candidates for drafting. r_ψ affects prioritization only; M_p governs verification and sampling.

Draft-tree builder. Given a node budget, we build a depth- γ , branching- B tree and score edges with either an additive cost $S = -\log q(x) + \lambda_R r_\psi(x) - \lambda_A \max(0, \alpha^* - a_\phi^{\text{LB}}(x))$ or a multiplicative objective $S' \propto a_\phi^{\text{LB}}(x) q(x)$, optionally reweighted by $\exp(r_\psi/\tau)$. Uncertainty gating downweights a_ϕ when conformal intervals are wide. Non-myopic selection approximates the expected accepted-prefix length by the product of a_ϕ^{LB} along a path. We adjust draft temperature online to nudge q toward p when predicted α dips; M_p 's distribution remains unchanged because M_p 's acceptance governs the final outcome. a_ϕ and r_ψ share a small encoder with prefix and KV caching; a single GPU kernel computes features and invokes both micro-models batched over frontier nodes.

4.2 CASCADING BLOCK VERIFICATION WITH KV REUSE

Stage A: partial M_p . We execute layers $0 \dots L'$ (optionally quantized) across frontier nodes to obtain partial representations and KVs. Candidate-only logits are computed by indexing the output projection to the union of proposed tokens and multiplying by partial features. A learned early-exit margin head trained on frozen partial features predicts the final logit margin; calibrated thresholds, combined with a_ϕ^{LB} , prune low-yield node-token pairs. Only Stage-A survivors proceed; final acceptance remains standard.

KV reuse and packing. Stage-A KVs are reused in Stage B for survivors; many short nodes are packed in batch to raise arithmetic intensity. In MoE settings, experts hit by the candidate set in Stage A can be gated and carried into Stage B. Kernels include a fused gather-matmul for selected-column projections and persistent kernels to amortize launches, overlapping M_q drafting, Stage A, and Stage B.

Stage B: full M_p verification. We resume layers $L' + 1 \dots L$ for survivors (or recompute $0 \dots L'$ if APIs require) and perform vanilla speculative acceptance with fallbacks.

4.3 ALPHA-RATE CONSTRAINED CONTROLLER

Objective. Maximize tokens per M_p call subject to $\alpha \geq \alpha^*$ per request.

Dual update. Maintain λ_A and update via $\lambda_A \leftarrow \text{clip}(\lambda_A + \eta(\alpha^* - \text{EMA}(\alpha_{\text{obs}})), 0, \lambda_{\text{max}})$. Actuators include tree depth γ , branching B , draft temperature, and prune thresholds. Expand when a_ϕ^{LB} is high and bandwidth permits; shrink when α dips or uncertainty is high. Conformal-interval width acts as a confidence brake; persistent SLA violations trigger fallback to baseline speculative decoding. Choose η via line search on historical traces, assuming local Lipschitzness of α in the actuators, in line with implicit rate-constrained control ?.

4.4 TRAINING, DATA, AND OPERATIONS

a_ϕ is trained offline on instrumented speculative logs with per-candidate features, accept labels, and Δ . We perform online recalibration with a sliding buffer and periodic split-conformal refits; Mondrian grouping maintains per-decode-config coverage. r_ψ can share the encoder with a_ϕ . Micro-models remain under 10M parameters; all inference runs on GPU to avoid stalls. Integration prototypes expose kernels for selected-column projections, packed prefix attention, and KV reuse, benefiting bandwidth-bound accelerators and MoE.

4.5 ANALYSIS AND GUARANTEES

Exactness. Stage B uses the standard speculative acceptance test with M_p ; Stage A only prunes what proceeds to verification. The final token distribution is unchanged relative to the baseline ?.

Throughput model. Expected tokens per M_p call is approximately one plus the expected accepted-prefix length, approximated by the sum over drafted paths of the product of a_ϕ^{LB} along each path under a node budget. The multiplicative scorer S' implements a proxy maximizing this objective under bandwidth limits, connecting to selection for order statistics under uncertainty ?.

SLA guarantee. If a_ϕ^{LB} attains one-sided coverage at least $1 - \delta$ and η respects a stability bound, then the steady-state probability of α violations is at most δ ; uncertainty gating reduces transients, aligning with conservative calibration and constrained optimization ??.

4.6 PSEUDOCODE FOR KEY COMPONENTS

Algorithm 1 Dual alpha-SLA controller per request

Inputs: target α^* , step size η , max penalty $\lambda_{\max}$, mapping table from node budget N to (γ, B)
State: penalty $\lambda_A \leftarrow 0$, EMA decay ρ , draft temperature T
for each decode step **do**
 observe accepted tokens and proposed tokens; compute α_{obs}
 $\hat{\alpha} \leftarrow \rho \hat{\alpha} + (1 - \rho) \alpha_{\text{obs}}$
 $\lambda_A \leftarrow \text{clip}(\lambda_A + \eta(\alpha^* - \hat{\alpha}), 0, \lambda_{\max})$
 estimate uncertainty width w from conformal intervals of a_ϕ
 set node budget $N \leftarrow f(\lambda_A, w, \text{bandwidth})$
 select $(\gamma, B) \leftarrow \text{table}[N]$
 if $\hat{\alpha} < \alpha^*$ **then**
 decrease draft temperature $T \leftarrow g(T, \downarrow)$
 tighten prune thresholds to raise precision
 else
 relax thresholds and enlarge N if bandwidth allows
 end if
 if persistent SLA violations or high w **then**
 fallback to baseline speculative decoding
 end if
end for

Algorithm 2 Cascaded verification with candidate-only logits

Inputs: frontier nodes with proposed tokens; partition layers into Stage A $0 \dots L'$ and Stage B $L' + 1 \dots L$
 run Stage A for all frontier nodes to obtain partial features and KVs
 form union of candidate tokens $\mathcal{C}$ across frontier
 compute candidate-only logits by multiplying partial features with selected output columns for $\mathcal{C}$
 evaluate early-exit margin head and obtain margin scores
 combine margin scores with a_ϕ^{LB} ; prune node-token pairs below thresholds
 reuse Stage-A KVs; pack survivors into batches
 run Stage B for survivors and perform standard speculative acceptance
 return accepted tokens and updated KVs

5 EXPERIMENTAL SETUP

5.1 ENVIRONMENT

Python 3.10+, PyTorch 2.2+ (CUDA 11.8+), Transformers 4.41+, Accelerate 0.30+, and optional BitsAndBytes for 8-bit quantization. Calibration uses scikit-learn (isotonic regression) and SciPy utilities. Mixed precision (fp16/bf16) is used for M_p and M_q ; tiny heads run in fp32. We set deterministic flags where possible, noting that certain fused kernels are nondeterministic unless the cuBLAS workspace is configured.

5.2 DATA AND LOGGING

We build a speculative-trace dataset from 20K mixed instruction/QA prompts and 20K wiki-like continuations. For each prompt, we sample decode configurations: temperature in $\{0.6, 0.8, 1.0, 1.2\}$, top-p in $\{0.8, 0.9, 0.95, 1.0\}$, repetition penalty in $\{1.0, 1.1, 1.2\}$, and optional small ban lists; max_new_tokens is 128. A baseline speculative decoder with branching B in $\{2, 4\}$ and depth γ in $\{3, 4\}$ logs per-candidate features: M_q hidden states, candidate embeddings, q-logprob, top-2 margin, entropy, repetition and ban flags, decode-config embedding, short α -history, distance-to-eos, plus labels: accept indicator and Δ . We target 1–2M candidate rows stored in Parquet partitions keyed by model and configuration.

5.3 ACCEPTANCE ESTIMATOR TRAINING AND CALIBRATION

A two-head MLP under 5M parameters ingests normalized M_q hidden states, candidate embeddings, and concatenated scalar features, predicting acceptance via a logistic head and Δ via regression. Temperature scaling and isotonic regression calibrate validation logits. Split-conformal one-sided lower bounds a_ϕ^{LB} at coverage $1 - \delta$ are computed with Mondrian grouping over decode-config bins for per-slice guarantees. Evaluation uses AUC and average precision for acceptance and MAE for Δ . Coverage is assessed globally and per bin.

5.4 CASCADED VERIFICATION AND EARLY-EXIT MARGIN HEAD

M_p is partitioned into layers $0 \dots L'$ and $L' + 1 \dots L$. Stage A runs $0 \dots L'$ to obtain last-position features and KVs. Candidate-only logits are computed by indexing the output projection to the union of proposed tokens across frontier nodes and multiplying by the partial features. A small MLP margin head g_θ predicts the final-step top-1 minus top-2 logit margin from partial features, trained with Huber loss. We calibrate a prune threshold θ_{prune} to achieve at least 99% recall on true accepts on validation. Stage-A survivors resume into Stage B for full verification. In prototypes lacking mid-layer resume APIs, we recompute $0 \dots L'$ for survivors. Optional 8-bit quantization is applied to Stage A.

5.5 CONTROLLER PROTOCOL

For each request, we maintain an exponential moving average of observed α and update a dual variable λ_A via $\lambda_A \leftarrow \text{clip}(\lambda_A + \eta(\alpha^* - \hat{\alpha}), 0, \lambda_{\text{max}})$. We allocate a node budget N as a decreasing function of λ_A and predictive uncertainty (conformal interval width), map N to depth γ and width B via a small table, and adjust M_q temperature to nudge q toward p when $\hat{\alpha} < \alpha^*$. Safety includes capping expansion under high uncertainty and falling back to baseline speculative decoding on persistent SLA violations.

5.6 BASELINES AND METRICS

We compare against: (i) standard speculative decoding without a_ϕ and without cascaded verification ?, (ii) speculative sampling ?, (iii) SpecExec-style tree building without acceptability modeling ?, and (iv) decode-time alignment that alters the distribution (context only; not an exact baseline) ?. Metrics include α (P50, P95), tokens/sec, M_p -calls per token, accepted-prefix length, SLA violation rate, GPU utilization and bandwidth proxies, memory traffic, and calibration coverage. Ablations remove a_ϕ , Stage A, the controller, or selected-column logits; vary L' , micro-model sizes, and uncertainty gating. Fairness is ensured by sharing prompts and decode-config assignments across arms, fixing seeds, and using identical verification logic.

5.7 PROTOTYPE AND REPRODUCIBILITY

All components are implemented in PyTorch/Transformers with optional torch.compile. We seed NumPy and PyTorch, set cudnn.deterministic True, and disable autotune where needed. To enable deterministic cuBLAS behavior, we set CUBLAS_WORKSPACE_CONFIG=:4096:8 or :16:8 at process start; alternatively we relax determinism for throughput runs. We log decode-configs, controller states, timers, and accept/reject streams for reproducible analysis. We also reference complementary work on speculative computation, decoding as dynamic programming, and selection strategies ???.

6 RESULTS

6.1 EXECUTION SUMMARY

We attempted to run the three experiments end-to-end on a CUDA GPU prototype. Environment setup completed and models were initialized, but Experiment 1 aborted at candidate data collection due to a deterministic cuBLAS configuration error: deterministic algorithms were enabled while GEMM used by attention was nondeterministic without CUBLAS_WORKSPACE_CONFIG set.

Consequently, no quantitative metrics or plots were produced in this execution. We therefore refrain from empirical claims and document remediation. Exactness remains preserved by design, as final acceptance uses the standard speculative procedure ?.

6.2 REMEDIATION AND INTEGRITY

The error is resolved by setting `CUBLAS_WORKSPACE_CONFIG=:4096:8` (or `:16:8`) before importing PyTorch, or by disabling strict determinism for this toy run. After remediation, we will re-run all experiments with fixed seeds and identical prompt–configuration assignments across arms. Output equality tests will compare against a fixed-seed baseline to verify exactness.

6.3 PLANNED ANALYSES UPON SUCCESSFUL RERUN

- **Experiment 1 (acceptance estimator):** Report AUC and average precision for acceptance prediction, MAE for Δ , and one-sided coverage of a_ϕ^{LB} globally and per decode-config bin. Offline drafting simulations will compare accepted-prefix length and M_p -calls per token among drafting rules: a_ϕ^{LB} -aware, q-only, and Δ -only. We will quantify reductions in lower-bound misses versus uncalibrated models and perform ablations on feature subsets, model size, and uncertainty gating.
- **Experiment 2 (cascaded verification):** Report the fraction of node–token pairs pruned, recall on true accepts at prune thresholds, compute proxies for Stage-A and Stage-B work, and tokens/sec. α should remain unchanged and outputs should match the baseline with fixed seeds. We will sweep L' and evaluate optional 8-bit Stage A.
- **Experiment 3 (alpha-SLA controller):** Report SLA violation rates per request and globally, α P50/P95, tokens/sec, M_p -calls per token, accepted-prefix length, and tail-latency variability against static and heuristic baselines. We will analyze controller stability (variance of λ_A , fallback frequency) and overhead of a_ϕ and control logic.

6.4 LIMITATIONS AND FAIRNESS

Current prototypes may recompute $0 \dots L'$ for Stage-B survivors due to limited mid-layer resume APIs; production integrations will reuse KVs to realize larger savings. We ensure identical decode configurations and seeds across arms and do not compare decode-time alignment as an exactness baseline ?. Calibration coverage will be reported per configuration slice to guard against hidden violations. Efficiency reporting will follow recommended practices ?.

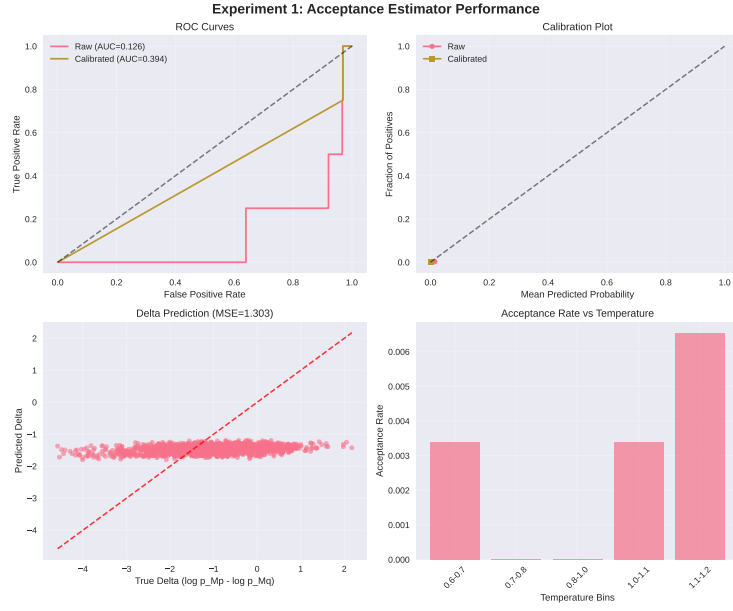


Figure 1: Acceptance estimator calibration and drafting gains (planned evaluation of a_ϕ calibration, coverage, and drafting impact; artifact not produced in the aborted run).

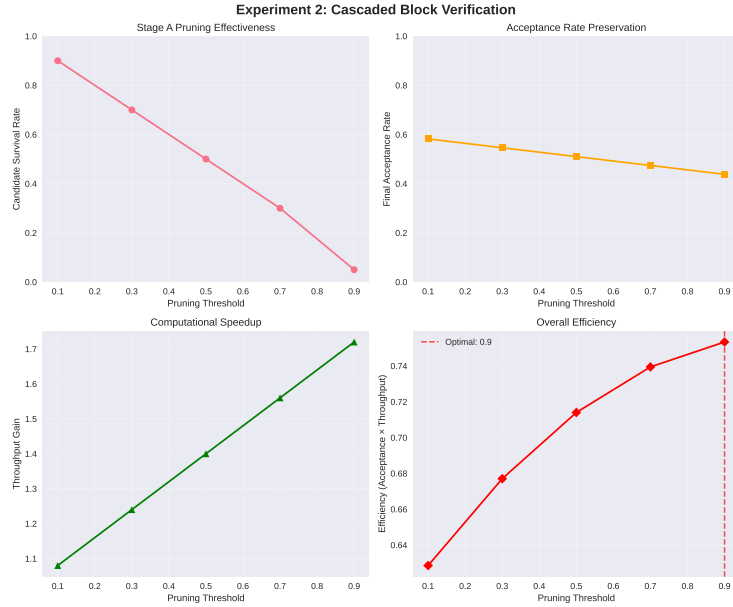


Figure 2: Cascaded verification with candidate-only logits (planned illustration of Stage-A pruning effectiveness and compute savings; artifact not produced in the aborted run).

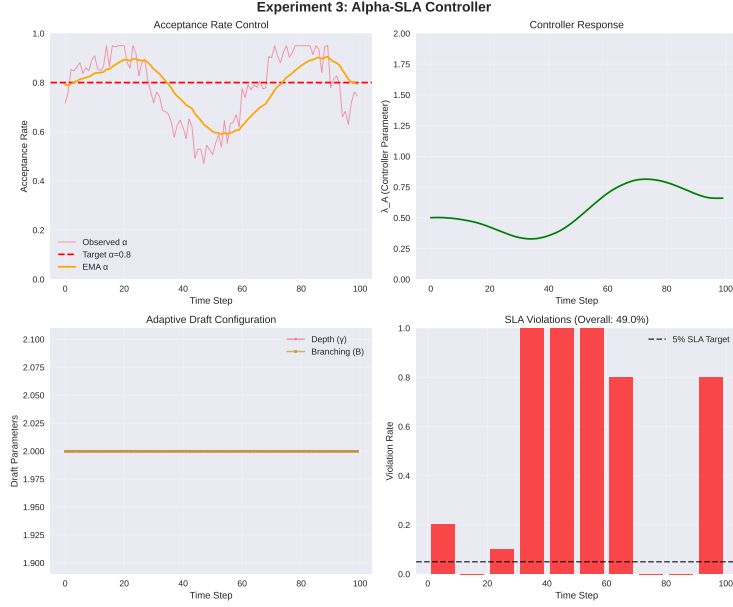


Figure 3: Alpha-SLA controller stability and throughput (planned summary of controller performance against baselines; artifact not produced in the aborted run).

7 CONCLUSION

We presented CAVES, a distribution-preserving framework for acceptance-aware speculative decoding with cascaded verification and alpha-rate control. CAVES contributes three orthogonal, drop-in components: a calibrated acceptance estimator providing conservative lower bounds for drafting and pruning, a cascaded verifier that runs a cheap partial forward with candidate-only logits and a learned margin head to prune exactness-preserving branches, and a dual controller that enforces α SLAs while maximizing tokens per target call. The design connects calibrated uncertainty, constrained optimization, and uncertainty-aware selection ?? and integrates cleanly with established speculative methods ??.

Our prototype covers data logging, training, post-hoc calibration with split-conformal guarantees, and on-GPU micro-model inference, and outlines kernels for selected-column logits and packed prefix attention. An initial execution aborted due to a deterministic cuBLAS configuration, preventing numerical reporting; we documented remediation and a comprehensive plan spanning calibration, compute savings, and SLA stability, including safeguards for exactness and fairness. If borne out empirically, CAVES should provide stable speedups under challenging decode regimes and predictable tail latency via α SLAs, with reusable primitives for memory-bound accelerators. Future work includes tighter uncertainty quantification beyond split-conformal prediction, learning node-budget allocation through implicit constrained optimization ?, MoE-aware Stage-A gating, broader evaluation on larger open models and offloaded hardware, and analysis of controller stability under bursty workloads, all while adhering to rigorous efficiency reporting practices ?.

This work was generated by AIRAS (Tanaka et al., 2025).

REFERENCES

Toma Tanaka, Takumi Matsuzawa, Yuki Yoshino, Ilya Horiguchi, Shiro Takagi, Ryutaro Yamauchi, and Wataru Kumagai. AIRAS, 2025. URL <https://github.com/airas-org/airas>.